

KI-Zusammenarbeit im CommandPilot-Projekt

Verbindliche Arbeitsweise fuer ChatGPT, Claude Web, Claude CLI und Codex CLI im CommandPilot-Projektordner

Kurzfassung: Ein Tool baut. Ein Tool reviewed. Ein Tool plant. ChatGPT haelt Strategie, Scope und Produktentscheidung zusammen.

1. Ziel dieses Dokuments

Dieses Dokument legt fest, wie die KI-Tools im CommandPilot-Projekt zusammenarbeiten. Es soll verhindern, dass mehrere Agenten gleichzeitig am Code arbeiten, Scope Creep entsteht oder Roadmap, Tickets und Deployment-Zustand unklar werden.

CommandPilot ist ein AI Morning Command Center und entwickelt sich langfristig zu einem persoentlichen Life- und Business-Operating-System. Kurzfristig ist das Ziel ein stabiler Private-Beta-Stand: Fresh User Flow, Login, Tagesstart, Tagesplan, Abend-Reflexion, Regeln, Sprache und Deployment muessen verlaesslich funktionieren.

2. Rollenverteilung

Rolle	Tool	Aufgabe	Nicht Aufgabe
Product Owner	Serkan	Finale Entscheidungen, Prioritaeten, Tests, Nutzerfeedback, Scope-Freigabe	Blind AI-Output uebernehmen oder mehrere Tools gleichzeitig bauen lassen
Strategy Advisor	ChatGPT	Strategie, Priorisierung, Scope-Kontrolle, Produktentscheidungen, Prompts, Review von Reviews	Direkt am Code arbeiten oder ohne Review neue Features freigeben
Projektmanager / UX Writer	Claude Web	Roadmap, Tickets, Akzeptanzkriterien, UX-Copy, FAQ, Onboarding- und Sales-Texte	Code aendern, neue Features in die Roadmap schmuggeln
Senior Developer	Claude CLI	Implementierung, Architektur, Refactors, Bugfixes, Backend/Frontend-Integration	Ohne Ticket-Scope frei weiterbauen
QA / Reviewer	Codex CLI	Code Review, Build-/Type-Checks, Security, Edge Cases, kleine klar definierte Fixes	Grosse Architekturentscheidungen oder parallel dasselbe Feature bauen

3. Grundregel: Nur ein Tool baut

Im CommandPilot-Projekt darf immer nur ein Tool produktiv am Code arbeiten. Andere Tools duerfen parallel planen, Review-Checklisten vorbereiten oder Texte schreiben, aber nicht denselben Codebereich veraendern.

Standardmodus: Claude CLI baut ein klar abgegrenztes Ticket. Codex reviewed danach den Diff. Claude Web pflegt Dokumentation, Copy oder Ticket-Spezifikation. ChatGPT entscheidet, ob weitergebaut, gefixt, deployed oder gestoppt wird.

4. Ticket-Workflow

Phase	Owner	Output
1. Scope klaeren	ChatGPT + Serkan	Ticket-Ziel, Nicht-Ziele, Akzeptanzkriterien, Risiko
2. Spezifikation	Claude Web	Ticketbeschreibung, Testplan, UX-/Copy-Anforderungen, falls noetig
3. Implementierung	Claude CLI	Code-Aenderungen nur fuer dieses Ticket, Summary, manuelle Tests
4. Review	Codex CLI	Approved / Not Approved, Must Fix, Risky but acceptable, Testliste
5. Entscheidung	ChatGPT + Serkan	Deployen, fixen, stoppen oder in Backlog verschieben
6. Deployment	Serkan	Migrationen ausfuehren, Build, Commit, Push, Render/Vercel, Smoke Test

5. CommandPilot-Projektstatus

Aktueller Stand: CommandPilot ist als Full-Stack-MVP gebaut und live deployed. Der aktuelle Fokus ist Private-Beta-Stabilitaet, nicht neue Feature-Breite.

Bereich	Status
Frontend	Next.js App Router, TypeScript, Tailwind, mobile Basis mit Bottom Navigation
Backend	FastAPI, Pydantic, Supabase, OpenAI, strukturierte Fehler
Auth	Login/Signup, stale/expired session handling, Fresh-User-Workspace-Hardening
Core Loop	Tagesstart -> AI Tagesplan -> Abend-Reflexion -> naechster Plan mit Review-Kontext
Sprache	Deutsch/Englisch ueber profiles.language, UI-Copy verbessert, DB-Constraint fuer en/de
Deployment	Vercel Frontend, Render Backend, Supabase DB/Auth

6. Aktuelle Prioritaeten

- Bestehenden Live-Stand stabil halten.
- Fresh-User-Flow mit Bruder/Freunden testen: Signup, Login, Tagesstart, Plan, Abend-Reflexion.
- Fehler und Reibung sammeln, nicht sofort neue Features bauen.
- Nur Blocker fixen, die Private Beta verhindern.
- Erst nach stabiler Nutzung naechstes Sprint-Ticket freigeben.

7. Was aktuell nicht gemacht wird

- Ticket 2 / Duplicate Guard nur starten, wenn explizit freigegeben.
- Keine Stripe-Integration.
- Keine Kalender-, Gmail-, Voice- oder Native-App-Integration.
- Keine Team-Funktionen, Analytics oder umfangreiche Onboarding-Flows.
- Keine parallelen Projekte wie MonteurPilot im CommandPilot-Code.
- Keine grossen UI-Redesigns waehrend Beta-Hardening.

8. Definition of Done fuer ein Ticket

- Ticket-Scope ist schriftlich klar und eng.
- Claude CLI hat nur relevante Dateien geaendert.
- Codex Review enthaelt keine Must-Fix-Punkte mehr.
- Build und Type-Check sind gruen oder akzeptierte Abweichung ist dokumentiert.
- Notwendige Supabase-Migrationen wurden im Repo abgelegt und live ausgefuehrt.
- Smoke Test auf Produktion wurde bestanden.
- Serkan hat das Verhalten selbst getestet oder bewusst als offen markiert.

9. Standard-Prompts

Claude CLI - Implementierung

Implementiere nur Ticket [ID]. Keine neuen Features. Keine unrelated Refactors. Nach der Umsetzung: geaenderte Dateien, Datenfluss, manuelle Tests und bekannte Risiken zusammenfassen.

Codex CLI - Review

Reviewe nur die Aenderungen fuer Ticket [ID]. Nicht implementieren. Pruefe Korrektheit, API-Vertraege, Security, Edge Cases, Build-Risiken und Scope Creep. Output: Approved / Not Approved, Must Fix, Risky but acceptable, Testliste.

Claude Web - PM / UX

Erstelle Roadmap-/Ticket-/UX-Copy-Unterlagen ohne Code. Keine neuen Features. Keine Scope-Erweiterung. Fokus: klare Begriffe, Akzeptanzkriterien, Testerkommunikation, FAQ, Onboarding- oder Sales-Texte.

ChatGPT - Strategie

Bewerte Reviews, Logs und Summaries streng. Trenne echte Blocker von Nice-to-have. Entscheide: fixen, deployen, stoppen oder spaeter. Erstelle enge Prompts fuer andere Tools.

10. Deploy-Checkliste

- Supabase-Migrationen ausfuehren und verifizieren.
- Backend: python -m compileall app.
- Frontend: npm run build.
- Commit mit klarer Message.
- Render Backend deployen und /api/health sowie /api/health/ai pruefen.
- Vercel Frontend deployen und Login, Sprache, Tagesstart, Plan und Abend-Reflexion testen.
- Fresh User Smoke Test mit neuem Account.
- Nach erfolgreichem Smoke Test keine neuen Features mehr am selben Abend starten.

11. Arbeitsregel fuer den Projektordner

Wenn ein neues Ticket gestartet wird, muss zuerst die Roadmap-PDF geprueft werden. Danach werden Scope, Nicht-Ziele, Claude-CLI-Prompt, Codex-Reviewprompt und manuelle Tests definiert. Ohne diese Vorbereitung wird kein Code geschrieben.

Version: v1.0 - fuer den CommandPilot-Projektordner